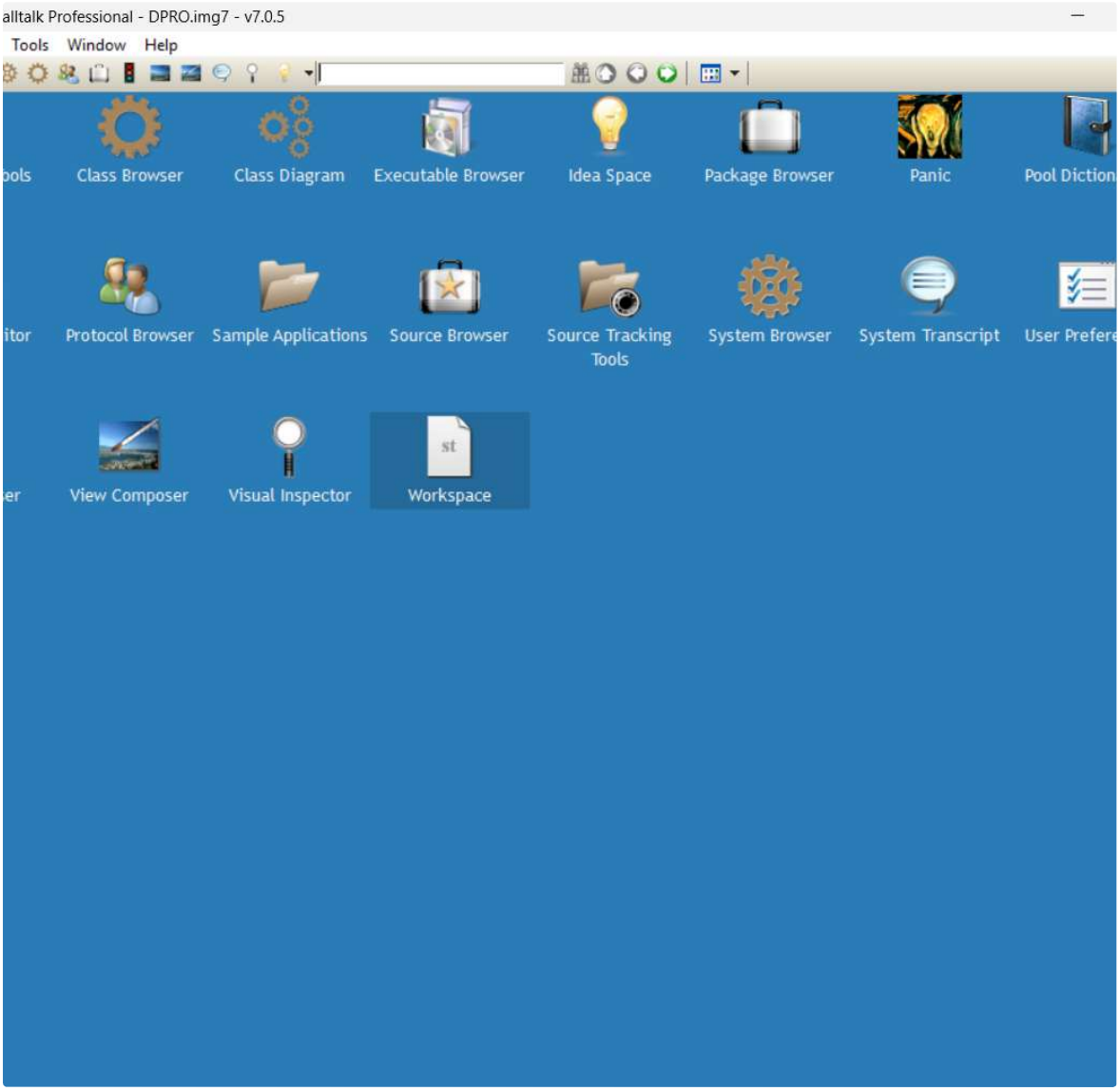
Programación Orientada a Objetos

(La posta de la materia esta aca) 😌

¿Qué es Dolphin Smalltalk? 🐬

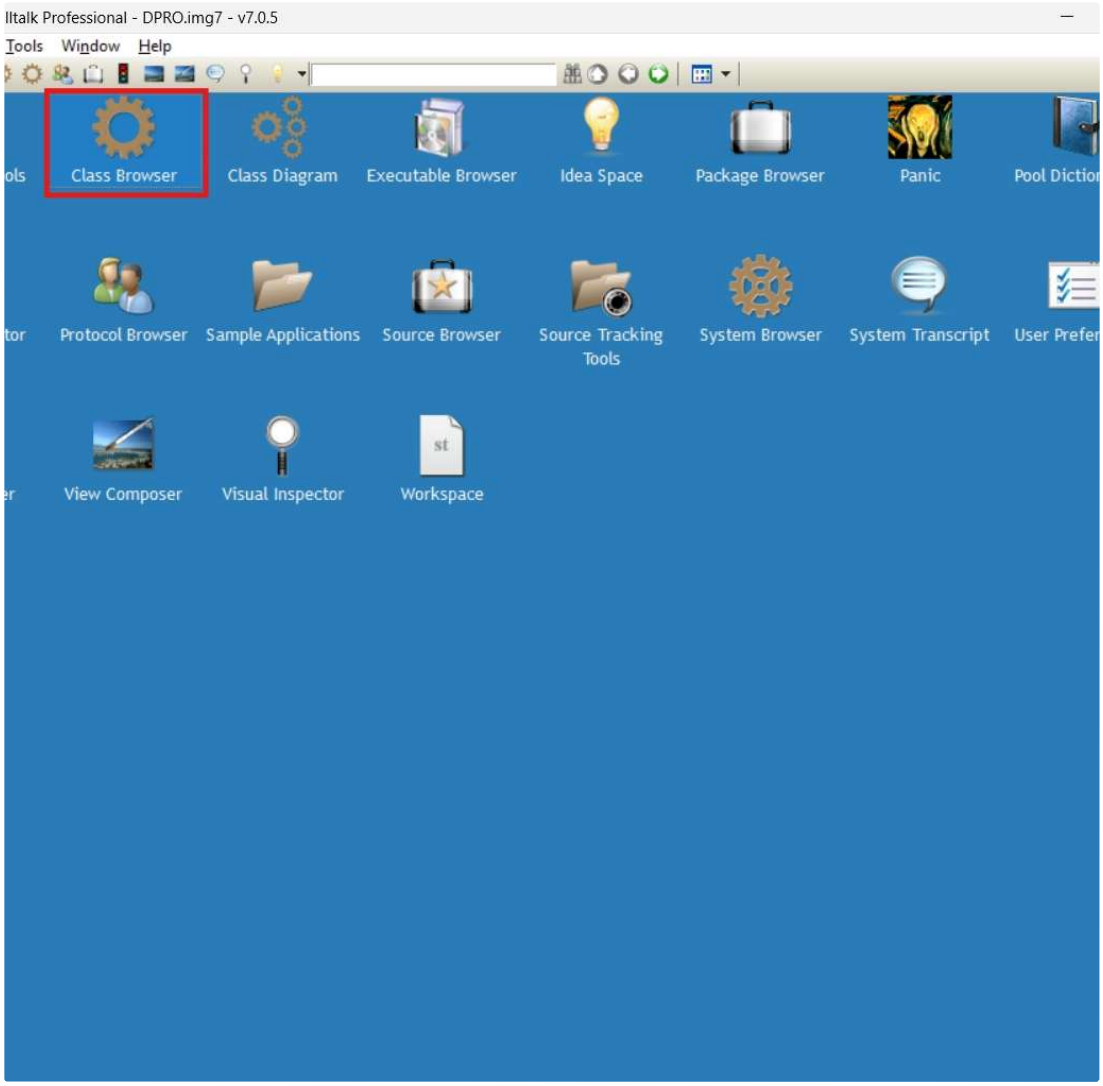
Si no sabes googlea, na mentira:
Dolphin Smalltalk es una implementación del lenguaje de programación orientado a objetos, que permite desarrollar aplicaciones tanto de consola como con interfaces gráficas.

Inicio del Dolphin



Lo que muestra la imagen, es la pantalla de inicio del entorno de Dolphin. En ella van a encontrar las diferentes herramientas con las que trabajamos en la cursada de paradigmas. Las principales son:

Class Browser ⚙️



El **Class Browser** es una herramienta que permite la exploración, creación y modificación de clases y métodos del sistema. Es la ventana donde se gestiona todo lo relacionado con la estructura de clases y métodos, convirtiéndose en el centro para programar y mantener código en Smalltalk.

En el Class Browser se puede:

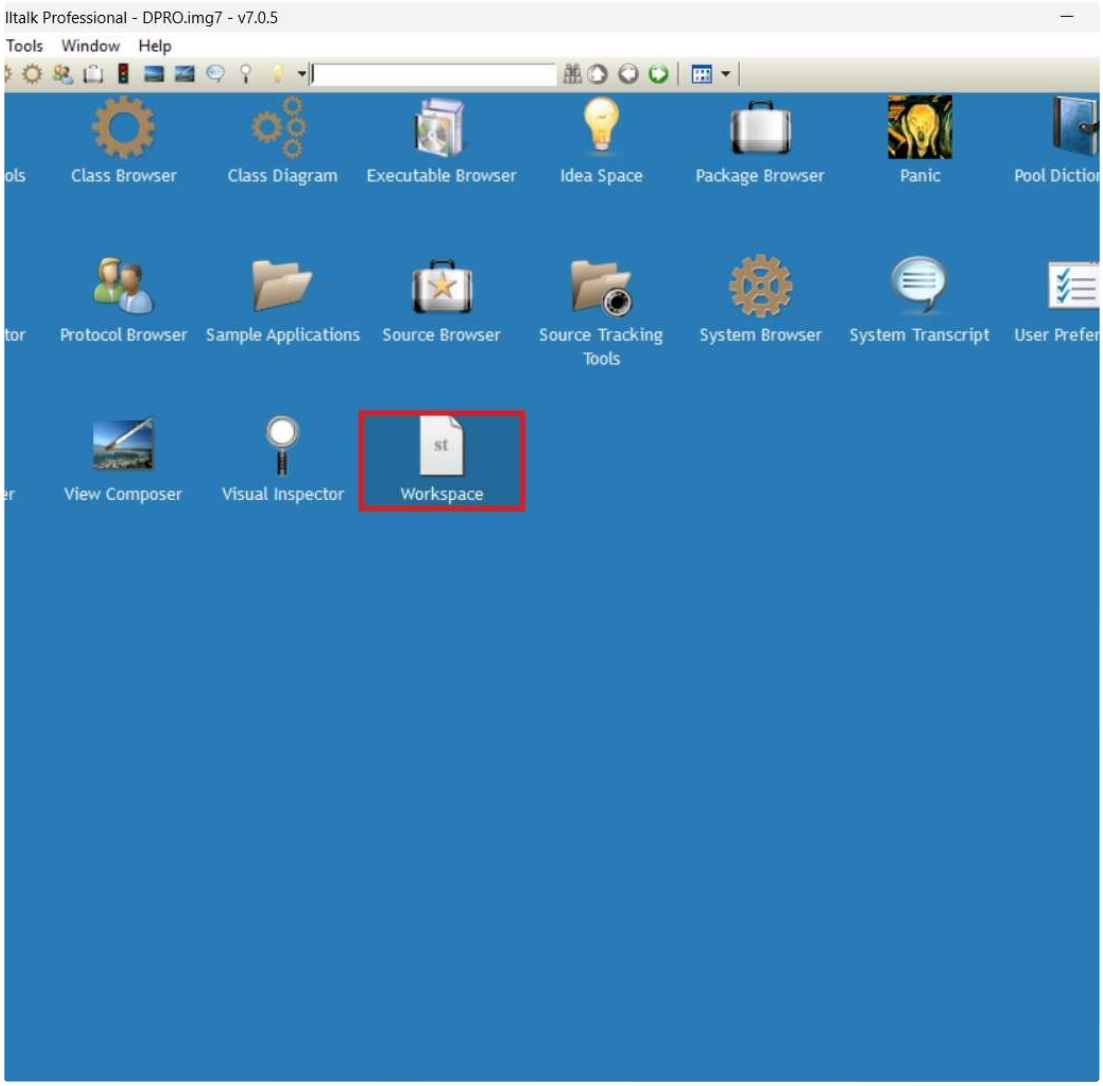
1. Navegar por el árbol de clases y jerarquías de herencia.
2. Crear nuevas clases o modificar las ya existentes.
3. Definir y editar métodos.

4. Acceder a la documentación y a ejemplos directamente vinculados con las clases.

¿Cómo crear Clases y métodos?

Por si no te diste cuenta esto de aca arriba es un link, podes hacer click para ver mejor que funionalidades tiene el **Class Browser** 💀💀💀

Workspace



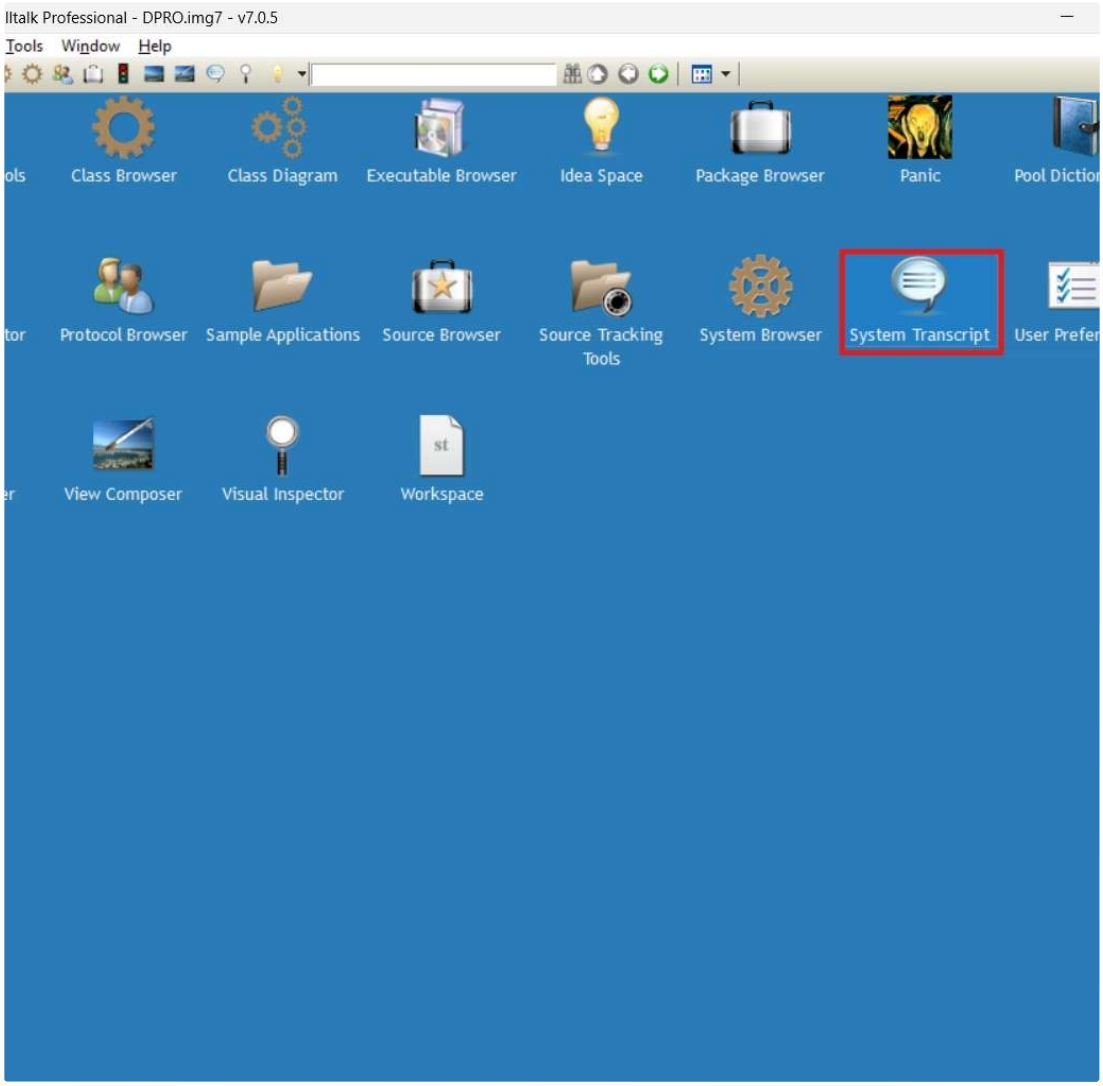
El **Workspace** (en español, *Espacio de trabajo*) es una ventana que permite la ejecución interactiva de código. Se trata de un entorno en el cual es posible crear objetos, enviarles mensajes, instanciar clases y realizar pruebas de manera inmediata para observar su comportamiento, todo esto sin necesidad de abandonar el entorno de desarrollo.

Este entorno es el espacio principal de trabajo durante la cursada, ya que en él se desarrolla el código necesario para la resolución del trabajo integrador.

Principales métodos, estructuras y utilidades del Workspace

Esto de acá arriba también es un link, hace click y vas a ver la magia 😉

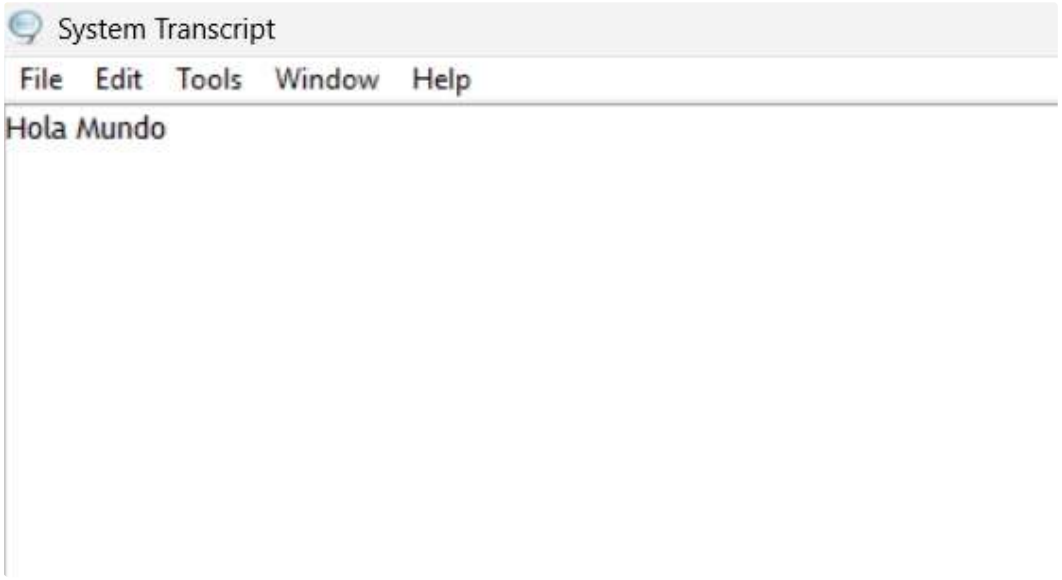
System Transcript 💬



El **System Transcript** es una ventana que funciona como consola de salida del sistema. Se utiliza para imprimir información de mensajes, resultados y notificaciones generados durante la ejecución de código en el Workspace.

Ejemplo:

Si dentro del Workspace se escribe `Transcript show: 'Hola Mundo'.` el resultado de la ejecución es el siguiente:

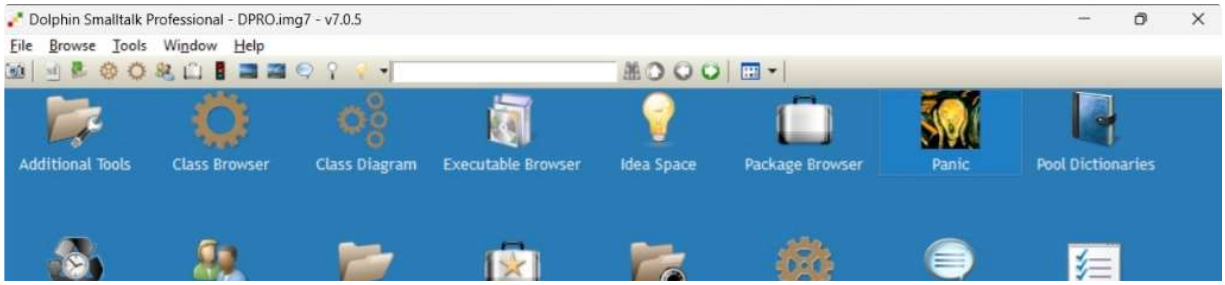


Este y demás ejemplos de la clase Transcript se desarrollan mejor en la sección "Principales métodos, estructuras y utilidades del Workspace"

Herramientas adicionales

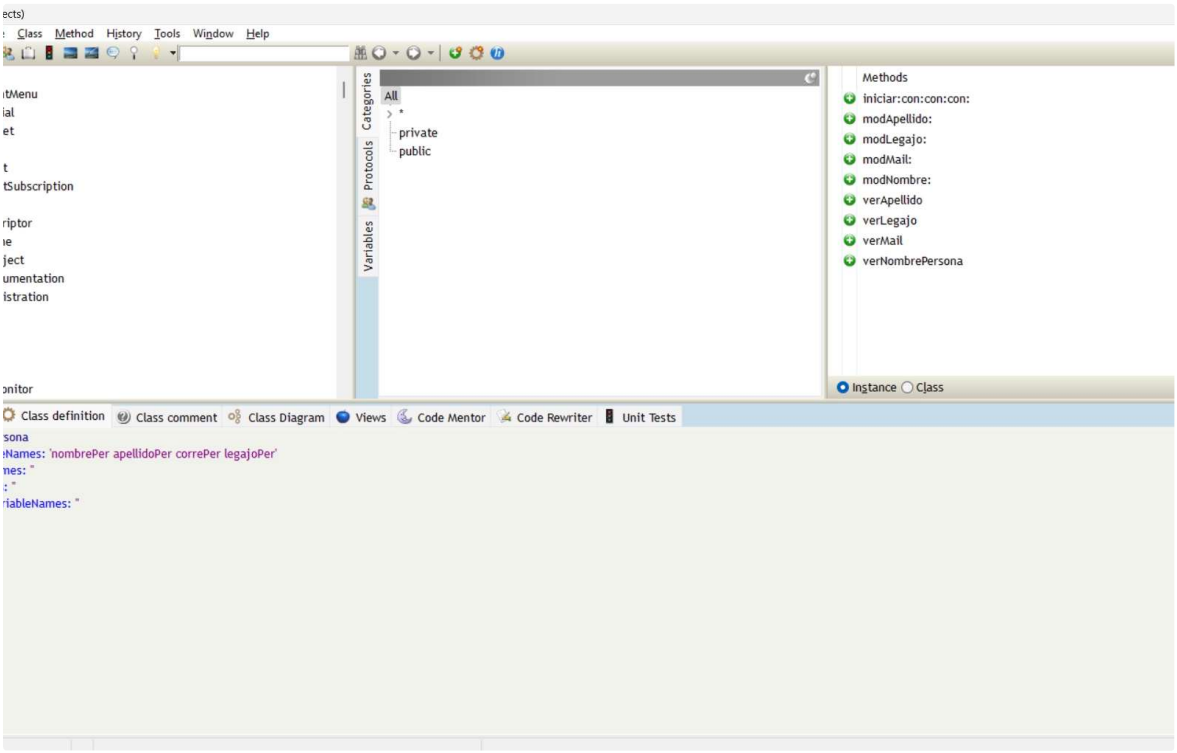
El resto de herramientas de Dolphin Smalltalk no se tienen en cuenta durante el desarrollo de la materia. Sin embargo, es válido mencionar algunas de ellas. Éstas son:

Visual Inspector 👓



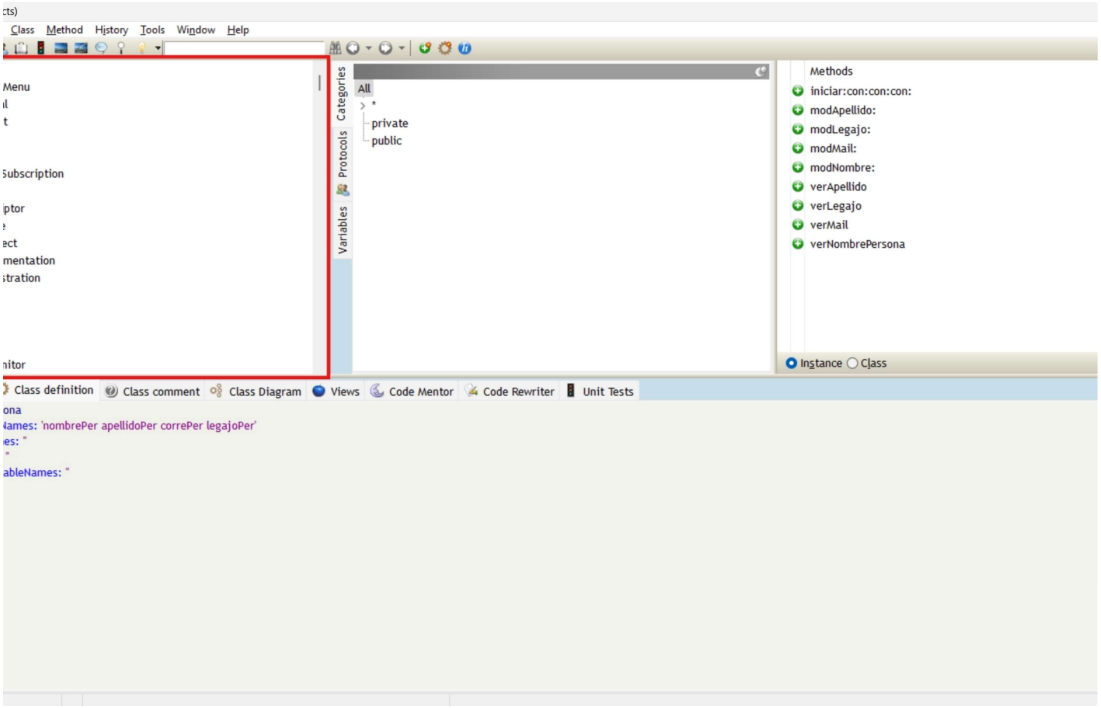
¿Cómo crear Clases y métodos? 🤯

Al hacer doble clic en **Class Browser**, se desplegará la siguiente interfaz.



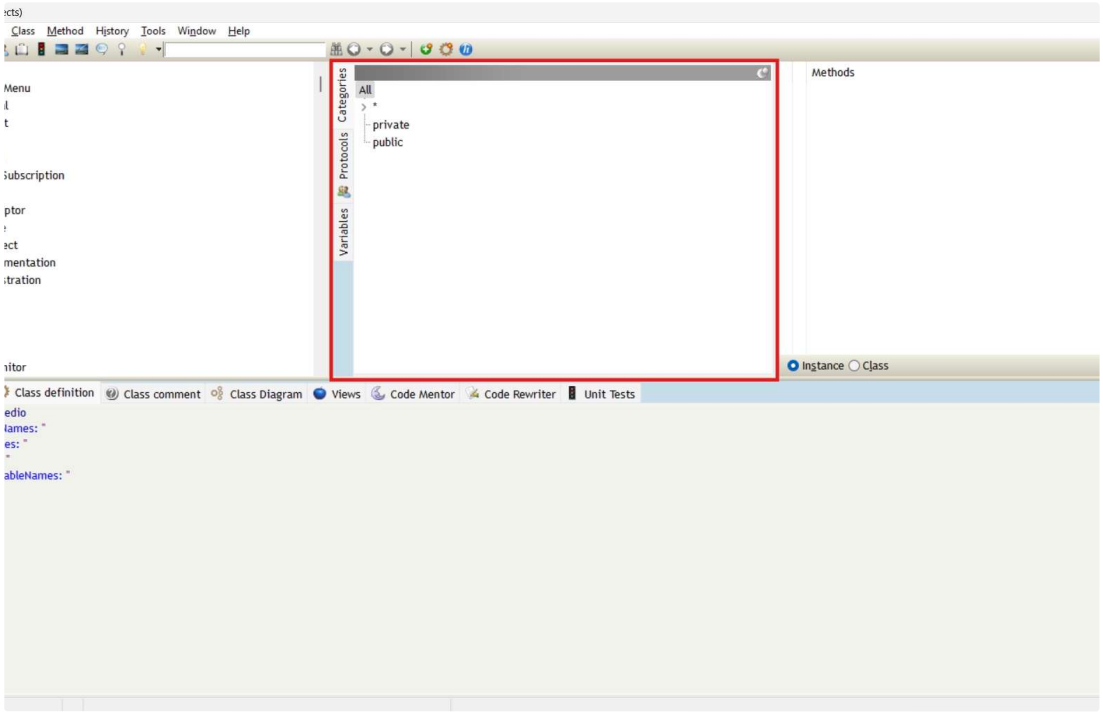
Antes de crear una Clase, es importante entender qué es lo que se ve en el panel del **Class Browser**. La imagen muestra cuatro secciones principales, que se describen a continuación.

1)Árbol de clases



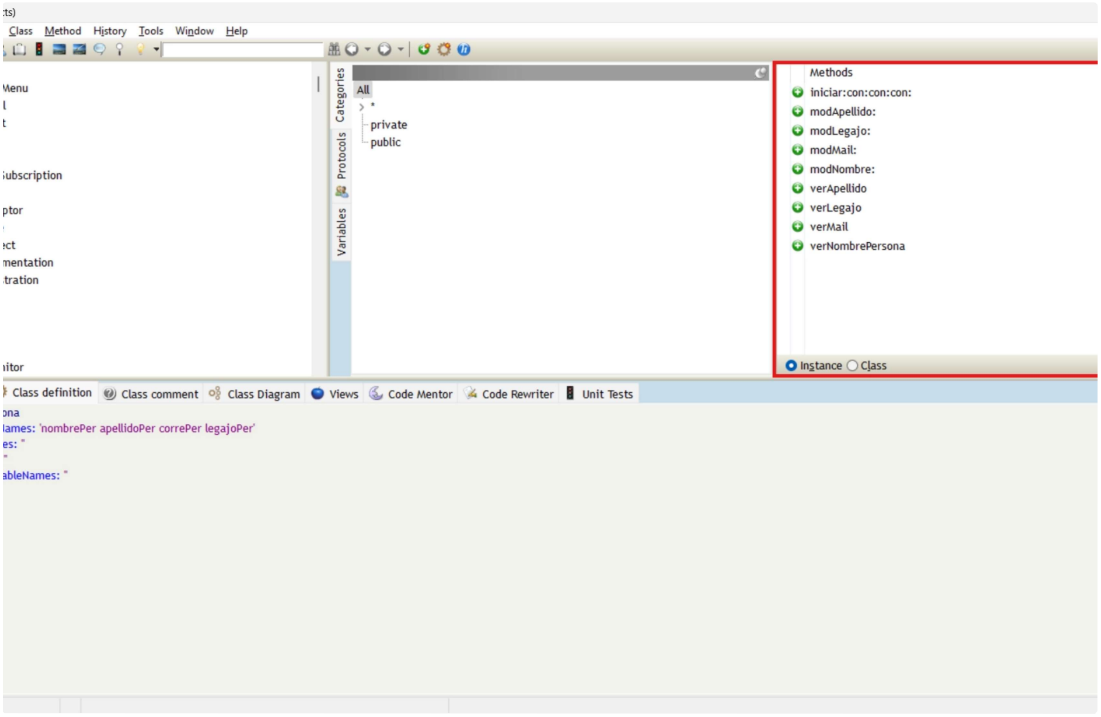
En este apartado se encuentran todas las clases existentes organizadas en una jerarquía de clases, destacándose una clase raíz denominada **Object**. Todas las clases nuevas, estarán contenidas dentro de **Object.**

2)Panel de categorías 🔓🔓



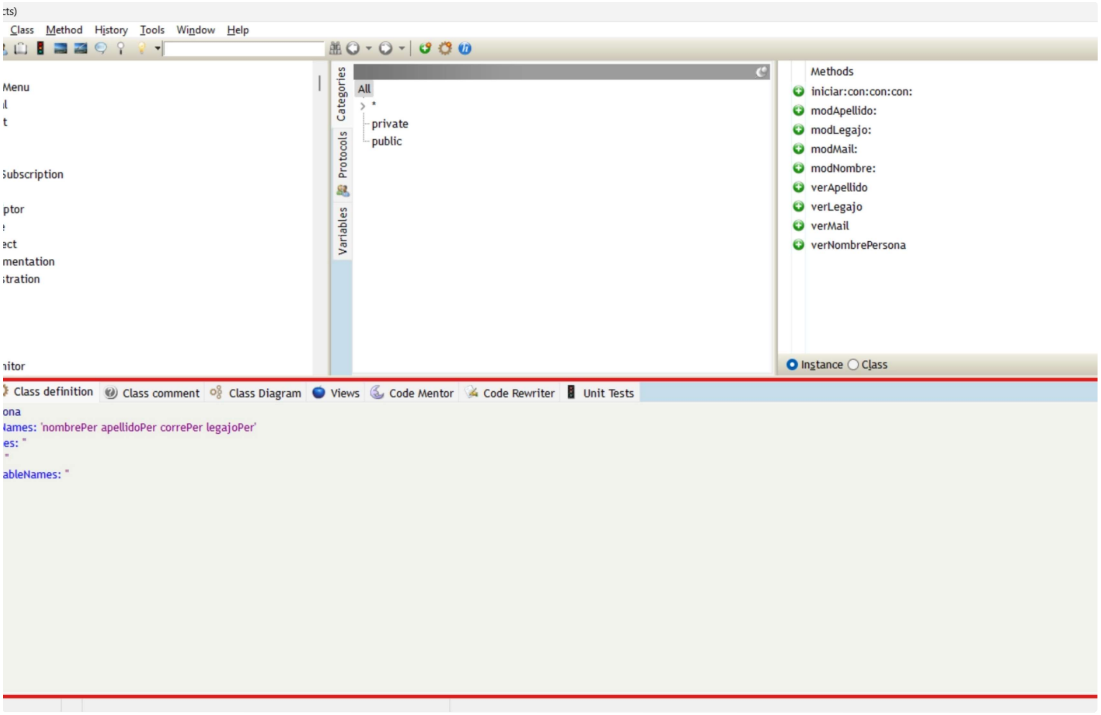
En este panel podemos explorar las **categorías de métodos** y las **variables** asociadas a la clase seleccionada. Desde aquí se puede acceder a los métodos públicos y privados, así como a las variables de instancia y de clase, facilitando la navegación y edición dentro del Class Browser.

3)Lista de métodos 📎



Esta sección muestra todos los métodos de Instancia o de Clase ya sean los ver, modificar, eliminar, agregar, etc

4)Definición de la Clase y Variables de Instancia

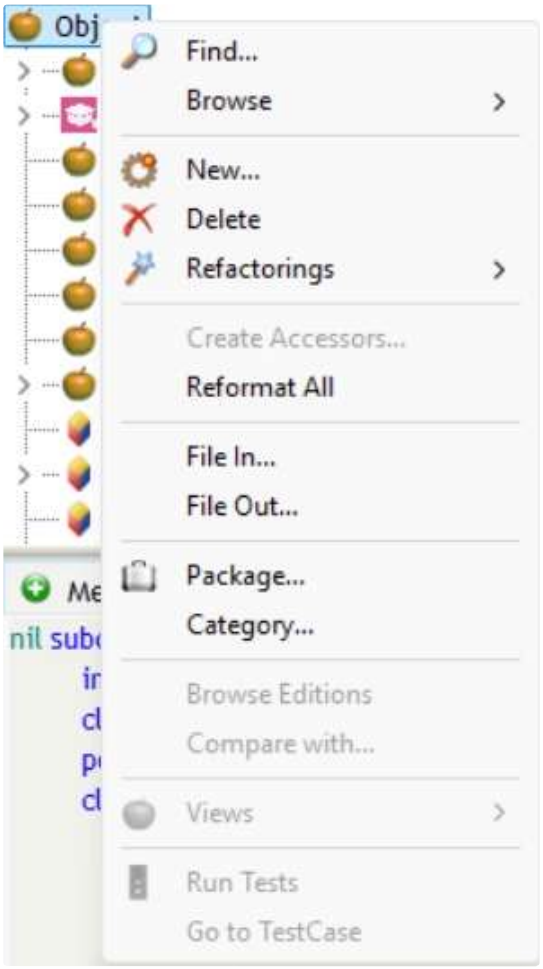


En este apartado vamos a poder definir como son las estructuras o el código de cada uno de los métodos ya sean de instancia o de clase, además nos permite poder definir cuales son los atributos de nuestra clase

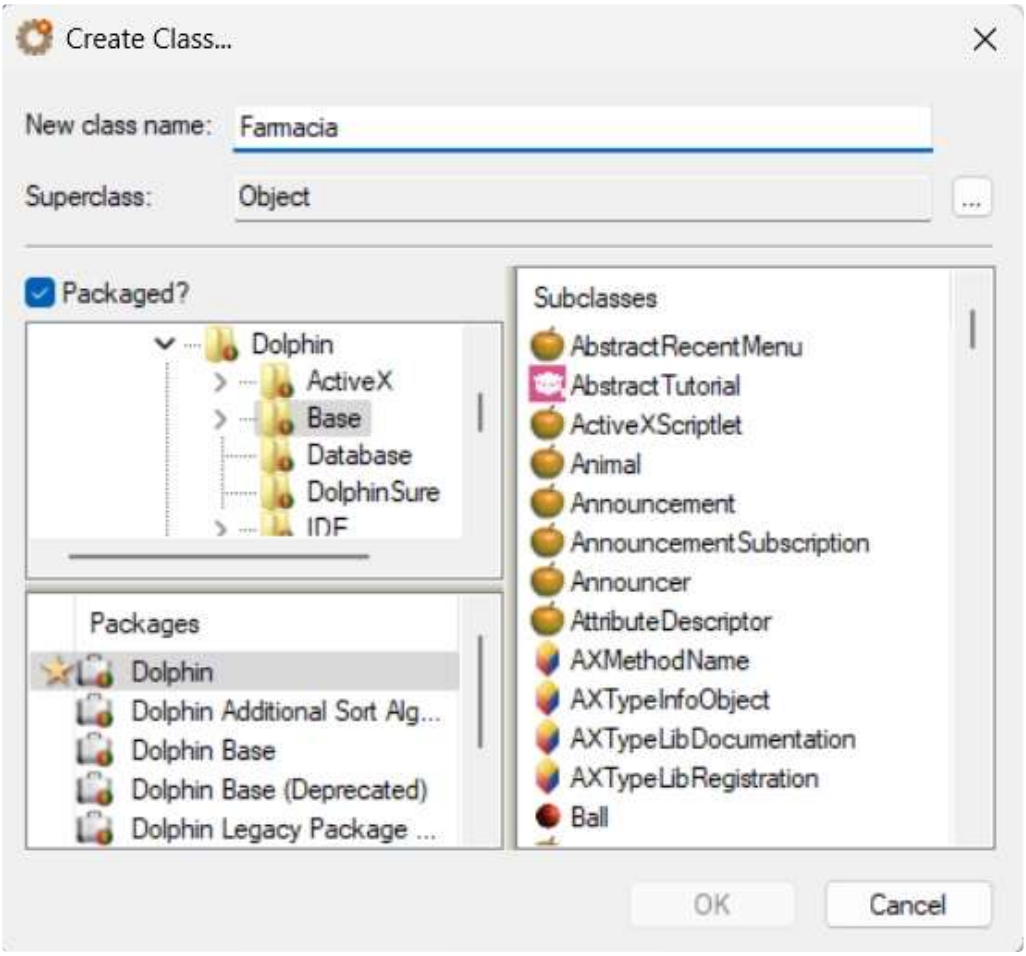
▼ Paso a paso para crear una clase con sus métodos



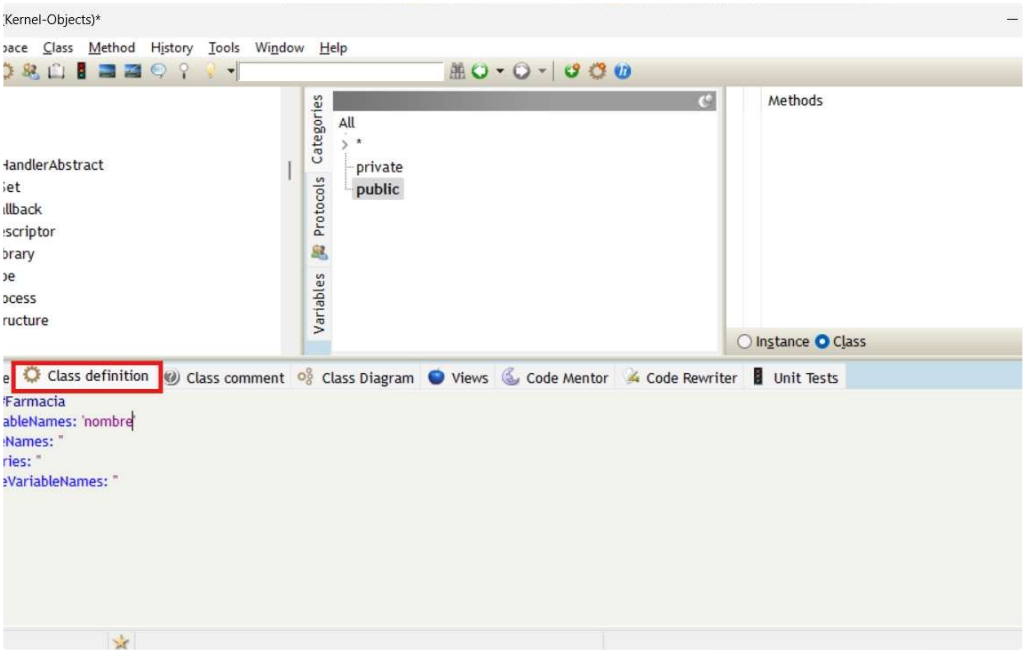
1. Realizar click derecho sobre Object, este se encuentra dentro de la sección árbol de clases y se va a desplegar el siguiente menu



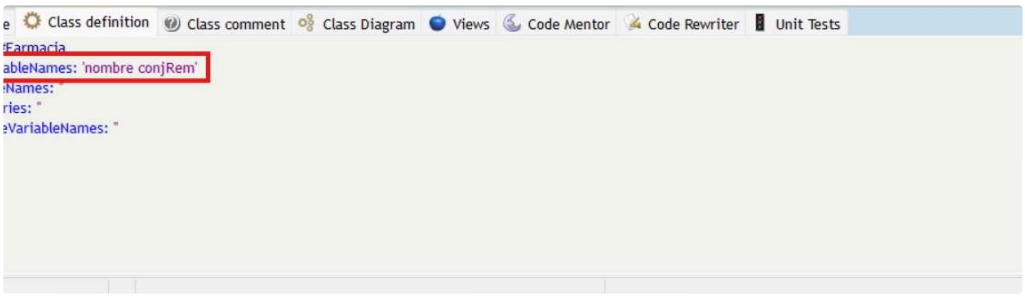
2. Seleccionamos “New” y vamos a ver la siguiente panel



3. Ponemos el nombre de la clase, **siempre con la primer letra en mayúscula** y le damos a OK
4. Luego nos dirigimos a “Class definition”

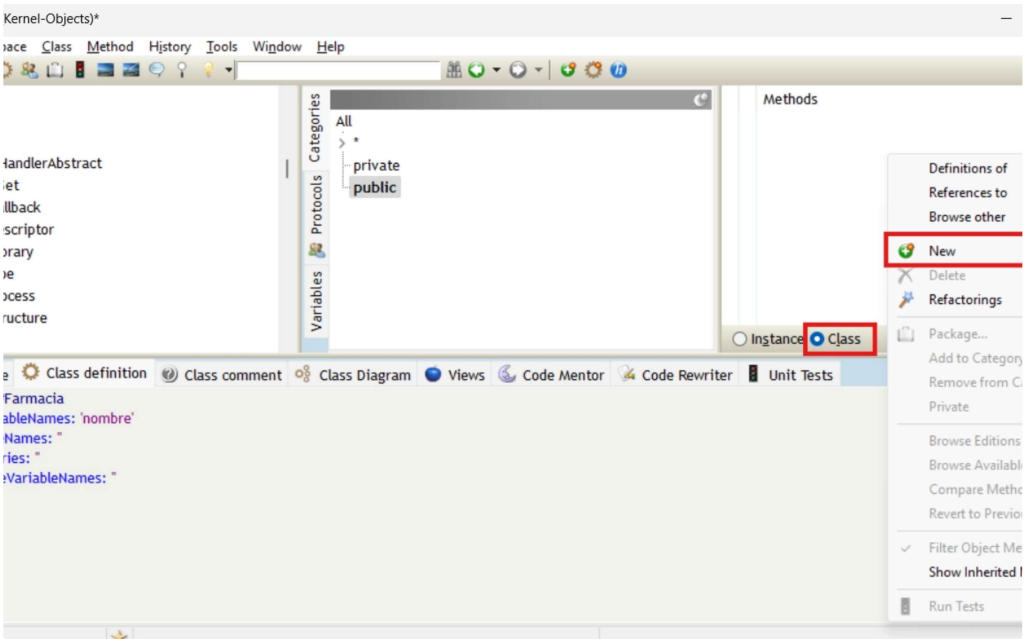


5. En el apartado de “instanceVariableNames:” vamos a agregar todos los atributos de la clase

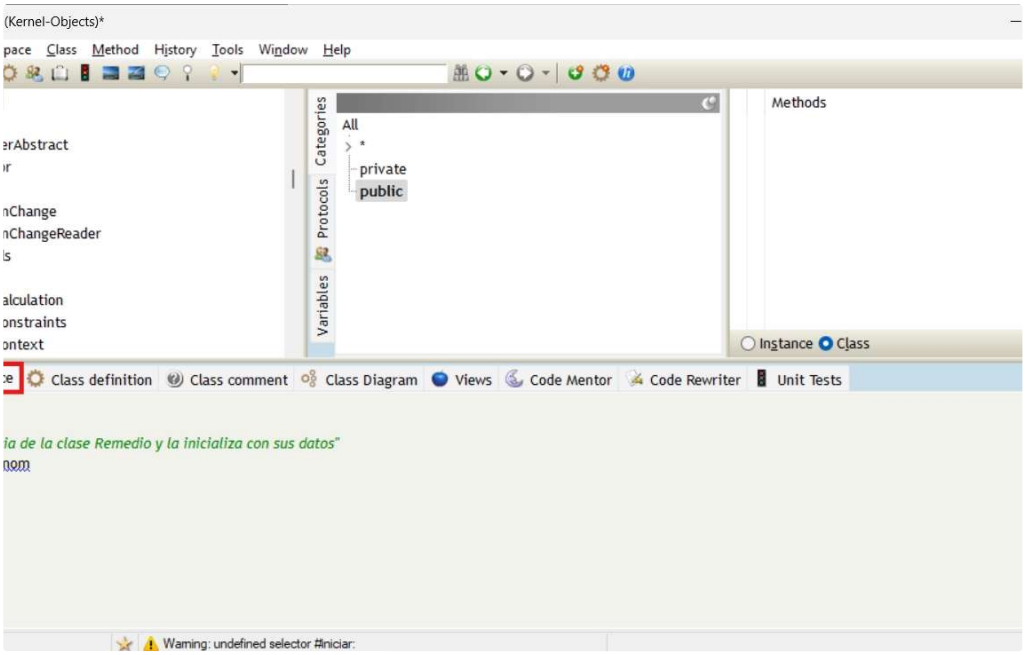


Nota: Si se crea una clase simple, no es necesario incluir el atributo conjRem. Este atributo solo se agrega en una clase compuesta.
En este caso, una Farmacia contiene una colección de remedios, por lo que se define el atributo conjRem, cuyo significado es conjunto de remedios.

6. Ahora vamos a crear el método de clase, para esto nos dirigimos a “Lista de métodos”(explicado arriba) y realizamos click derecho y seleccionamos “New”
Nota: Tener en cuenta que este seleccionado Class (con el circulo azul)



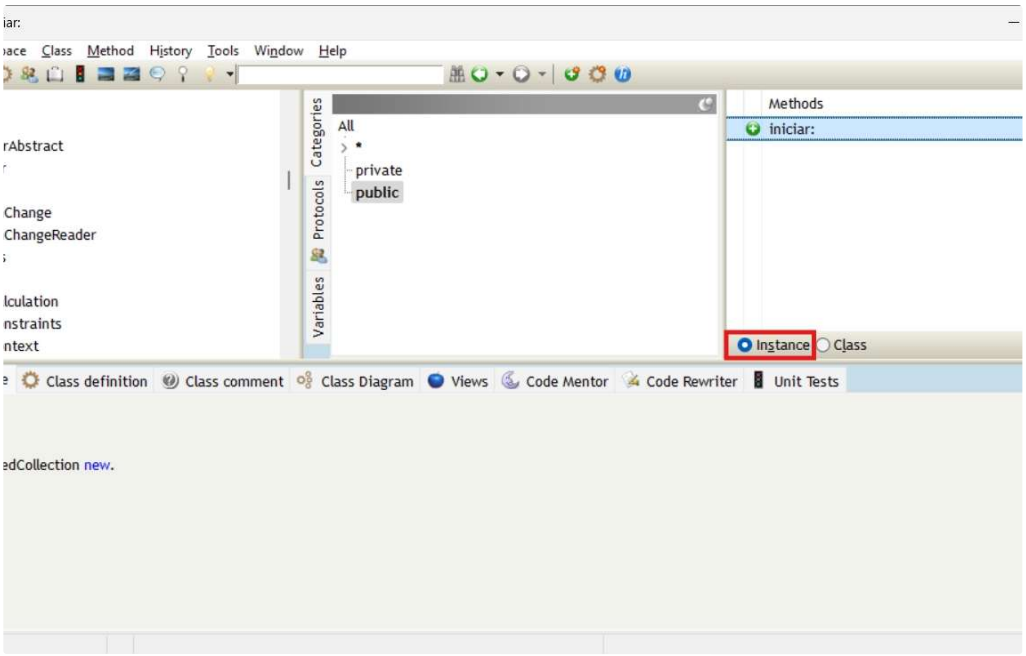
7. Una vez que seleccionamos “New” nos va a dirigir a “Method source” dentro de el vamos a crear el método de clase “crear” de la siguiente manera



Nota: Tengan en cuenta que se crea con todos los atributos definidos en el punto 5, en este caso tenemos solo el nombre, si llegamos a tener mas atributos lo podemos separar con la abreviación “con:” y el nombre de la variable que va a recibir, también en valido mencionar que la variable tiene que ser distinta a como se definió en Class definition, no podemos poner atributo “nombre” y variable “nombre”, es preferible poner la variable “unNombre”.
Ejemplo de remedio que tenia como atributos nombre, precio, stock, laboratorio
crear:nom con:unPre con:unStock con:unLab
^self new iniciar:nom con:unPre con:unStock con:unLab

8. Como se observa en la imagen anterior, la parte “**^self new iniciar: nom**” aparece subrayada en azul. Esto sucede porque aún no definimos el método iniciar dentro de los **métodos de instancia**. El siguiente paso será crearlo
9. Para guardar los cambios simplemente hacemos “Ctrl + S”

10. Seleccionamos Instance dentro del apartado “Lista de metodos” y nuevamente repetimos el paso 6



Nota: a diferencia de un objeto, los métodos **SIEMPRE arrancan en minúscula**, esto es para tenerlo en cuenta al crear los métodos, se reserva la primer letra mayúscula para los Objetos

▼ Creación de los métodos clase simple

1. **iniciar**
   El método iniciar se encarga de preparar un objeto recién creado, asignándole sus datos iniciales y dejándolo listo para ser utilizado. También es necesario definir que conjRem será una OrderedCollection, es decir, una colección ordenada donde se guardarán los remedios. Esto se hace inicializándola con OrderedCollection new.
   Ejemplo iniciar:nom



2. ...

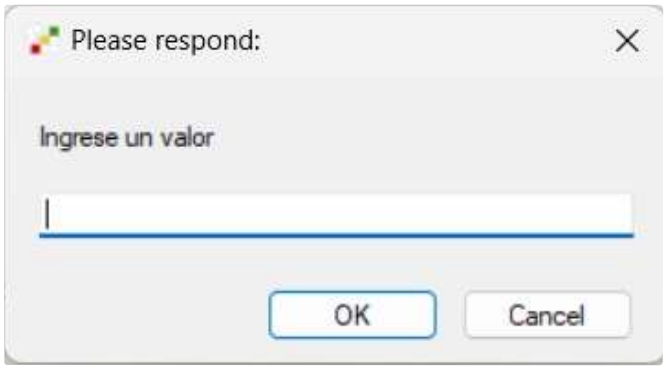
Principales métodos, estructuras y utilidades del Workspace 👍

▼ Prompter 🧙‍♂️

El Prompter es una clase que permite al usuario poder ingresar un valor por pantalla, se expresa de la siguiente manera.

```
Prompter prompt: 'Ingrese un valor'.
```

Una vez ejecutado ese comando, se ve de la siguiente manera



Una vez comprendido el funcionamiento del **Prompter**, es necesario definir un mecanismo para almacenar el valor ingresado por el usuario. Una alternativa es asignar dicho valor a una variable, lo cual puede realizarse de la siguiente manera:

```
var:=Prompter prompt: 'Ingrese un valor'.
```

En este caso “var” va a guardar el valor que se ingrese por el Prompter

¿Que pasa si queremos guardar un valor numérico?

En este caso, es necesario realizar una conversión utilizando el método `asNumber`, que permite transformar la cadena de tipo string en un número.

Se implementa de la siguiente manera

```
var:= (Prompter promt: 'Ingrese un valor')asNumber.
```

Transcript

El Transcript es un clase que permite enviarle mensajes a la ventana System Transcript (Explicada en Paradigmas). Existen distintos métodos de la clase Transcript, nosotros en la cursada vamos a ver los siguientes.

show: Este método te permite mostrar un texto dentro del System Transcript

```
Transcript show: 'Hola mundo'.
```

clear: Este método te permite borrar todo el contenido dentro del System Transcript

```
Transcript clear.
```

cr: Este método te permite realizar un salto de linea dentro del System Transcript

```
Transcript cr.
```

Conversión de datos

Dependiendo del contexto, podemos llegar a necesitar convertir de un tipo de dato a otro. Cuando necesitamos convertir una cadena de caracteres a un número para imprimirlo en el Transcript, lo podemos hacer con el método `displayString.`

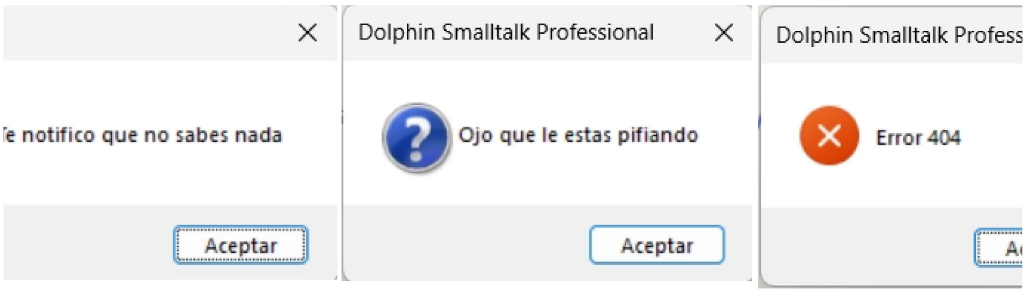
```
Trasncript show: (remedio verPrecio)displayString.
```

MessageBox 📦

El **MessageBox** es una clase que permite mostrar mensajes mediante una ventana emergente. Existen dos tipos:

- Sin valor de retorno: `notify`, `warning` y `error`. Su objetivo es solo informar al usuario, mostrando un mensaje con distinta estética según el tipo de aviso.

```
MessageBox notify: 'Te notifico que no sabes nada'.
MessageBox warning: 'Ojo que le estas pifiando'.
MessageBox errorMsg: 'Error 404'.
```



- Con valor de retorno: `confirm` A diferencia de los anteriores, devuelve un valor booleano (`true` o `false`) según la respuesta del usuario, lo que permite usarlo dentro de funciones o tomar decisiones en el programa.

```
MessageBox confirm: '¿Queres ingresar otro alumno?'
```

Como guardar el valor booleano en una variable

```
rta:= MessageBox confirm: '¿Queres ingresar otro alumno?'
```



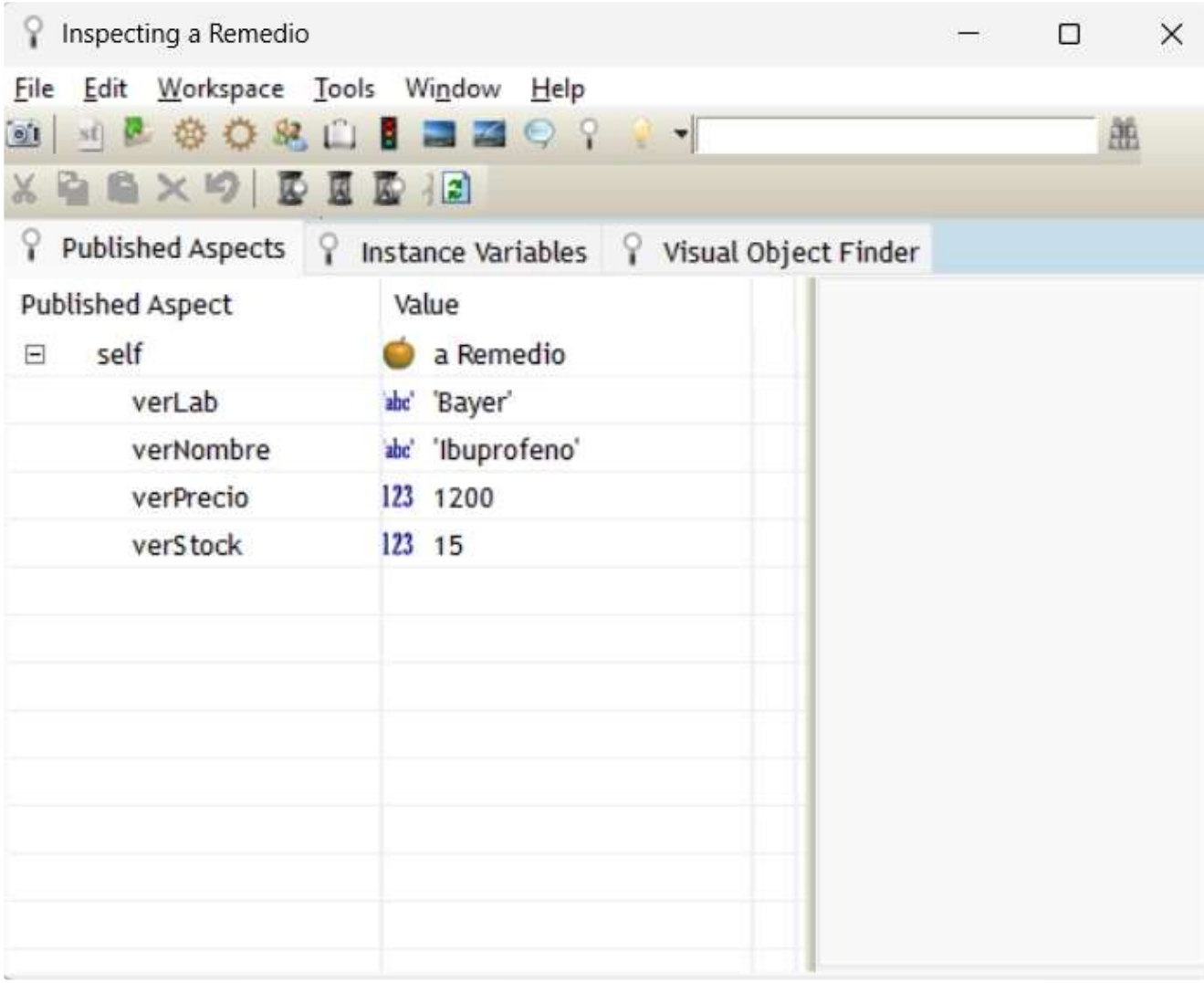
Les dejo una pregunta, que pasa si dentro de la condición del while pongo
"MessageBox confirm: '¿Queres cargar otro alumno?'"
Algo interesante puede hacer, seguro me ahorra un paso y una variable no se 🤔

Inspect

Esta herramienta nos permite a los programadores poder ver que esta pasando dentro de un objeto en tiempo real, ojo a la aclaración, esto es para el programador y **NO para el usuario**
Ejemplo
A nuestra clase remedio le cargamos un nombre, precio, stock y laboratorio como se muestra abajo

```
r1 := Remedio crear: 'Ibuprofeno' con: '1200' asNumber con: '15' asNumber con: 'Bayer'.
r1 inspect.
```

Luego de realizar el inspect, se visualiza la siguiente pantalla donde muestra todos los datos cargados.



En primera linea se visualiza el self que retorna un a Remedio en nuestro caso es la Clase Remedio
Luego todos los atributos con sus respectivos valores
Si tiene un 'abc' quiere decir que guardo un dato de tipo char
Si tiene un 123 quiere decir que guardo un dato de tipo numerico (por el asNumber)

▼ Estructuras de decisión ⚽

En **Dolphin Smalltalk**, las estructuras de decisión se basan en **mensajes enviados a valores booleanos (true o false)** y se utilizan **bloques de código** para definir qué hacer en cada caso.

(condición) `ifTrue:` [bloque de código].
(condición) `ifFalse:` [bloque de código].
(condición) `ifTrue:` [bloque de código] `ifFalse:` [bloque de código].

Nota: En caso de estructura de decisión la condición va entre paréntesis "()"

▼ Estructuras de repetición 🧉

En **Dolphin Smalltalk**, los bucles también se basan en **mensajes enviados a booleanos**, y los más comunes son **whileTrue:** y **whileFalse:**.
[condición] `whileTrue:` [bloque de código].
[condición] `whileFalse:` [bloque de código].

Nota: En caso de estructura de repetición la condición va entre corchetes "[ ]"

▼ Métodos de clase e instancia 🔟

Una vez creada una Clase (explicado en ¿Cómo crear Clases y métodos?) vamos a ver cuáles son sus métodos más comunes dentro del Workspace

▶ Métodos de clase

▶ Métodos de instancia

Iteradores de colección 🥅

Un iterador de colección en Dolphin Smalltalk es un mecanismo que permite recorrer secuencialmente los elementos de una colección.
De esta forma, se puede procesar, filtrar o transformar los datos sin necesidad de manejar índices manualmente.

- do
- reject
- select
- collect
- detect

Menú de opciones 🍔

Una herramienta muy útil para realizar la aplicación es el menú, es una manera mas sencilla e interactiva de poder mostrarle opciones al usuario

Su estructura es muy sencilla, simplemente utiliza un ChoicePrompter choices:

